

Lab: Basic Values, Variables, and Judgments

Learning Goals

The primary learning goals of this assignment are to build intuition for the following:

- the distinction between concrete and abstract syntax;
- the relationship between judgment forms/inference rules/judgments and implementation code;
- using a reference implementation as a definition of semantics;
- variable binding and variable environments.

Functional Programming Skills Recursion over abstract syntax. Representation invariants.

Programming Language Ideas Inductive definitions (grammars/productions/sentences and judgment forms/inference rules/judgments). Semantics (via detective work).

Instructions

A version of project files for this lab resides in the public [pppl-lab2](#) repository. Please follow separate instructions to get a private clone of this repository for your work.

You will be replacing ??? or `case _ => ???` in the `Lab2.scala` file with solutions to the coding exercises described below.

Your lab will not be graded if it does not compile. You may check compilation with your IDE, `sbt compile`, or with the “sbt compile” GitHub Action provided for you. Comment out any code that does not compile or causes a failing assert. Put in ??? as needed to get something that compiles without error.

You may add additional tests to the `Lab2Spec.scala` file. In the `Lab2Spec.scala`, there is empty test class `Lab2StudentSpec` that you can use to separate your tests from the given tests in the `Lab2Spec` class. You are also likely to edit `Lab2.worksheet.sc` for any scratch work. You can also use `Lab2.worksheet.js` to write and experiment in a JavaScript file that you can then parse into a JavaScript AST (see `Lab2.worksheet.sc`).

If you like, you may use this notebook for experimentation. However, **please make sure your code is in Lab2.scala; this notebook will not be graded.**

Recall that you need to switch kernels between running JavaScript and Scala cells.

Interpreter: JavaScripty Calculator

In this lab, we extend JavaScripty with additional value types and variable binding. That is, the culmination of the lab is to implement an interpreter for the subset of JavaScript with numbers, booleans, strings, the undefined value, and variable binding.

```
trait Expr // e ::=
case class N(n: Double) extends Expr // n

type Env = Map[String, Expr]
val empty: Env = Map.empty
```

```
def eval(env: Env, e: Expr): Expr = ???
```

We leave the `Expr` inductive data type mostly undefined for the moment to focus on the type of `eval`.

First, observe that the return type of `eval` is an `Expr` (versus `Double` in the previous lab), as we now have more value types. However, `eval` should return an `Expr` that is a JavaScripty value (i.e., is an $e : \text{Expr}$ such that `isValue(e)` returns `true`). The need for the object-language versus meta-language distinction is even more salient here than in the previous lab. For example, it is critical to keep straight that `N(1.0)` is the Scala value representing the JavaScripty value `1.0`.

Second, observe that the `eval` function takes a JavaScripty value environment `env: Env` to give meaning to free JavaScripty variables in `e`.

These ideas take unpacking, so let us start from the arithmetic sub-language of JavaScripty:

```
case class Unary(uop: Uop, e1: Expr) extends Expr // e ::= uop e1
case class Binary(bop: Bop, e1: Expr, e2: Expr) extends Expr // e ::= e1 bop e2

trait Uop // uop ::=
case object Neg extends Uop // -

trait Bop // bop ::=
case object Plus extends Bop // +
case object Minus extends Bop // -
```

```
case object Times extends Bop // | *
case object Div extends Bop   // | /
```

Thus far, we have considered only one type of value in our JavaScripty object language: numbers n (which we have considered double-precision floating-point numbers). Specifically, we have stated the following:

$$\begin{array}{lcl} \text{expressions } e & ::= & v \\ \text{values } v & ::= & n \end{array}$$

Recall from our preliminary discussion about evaluation ([?@sec-evaluation-preliminaries](#)) that the computational model is a rewriting or reduction of expressions until reaching values. A value is simply an expression that cannot be reduced any further. Thus, we can also consider a unary judgment form $e \text{ value}$ that judges when an expression is a value.

$$\frac{\text{NUMVAL}}{n \text{ value}}$$

This judgment form corresponds to the `isValue` function:

```
def isValue(e: Expr): Boolean = e match {
  case N(_) => true
  case _   => false
}
```

From our discussion on grammars and inference rules ([?@sec-grammars-and-inference-rules](#)), we can see this unary judgment form $e \text{ value}$ as defining a syntactic category values v . Thus, we freely write grammar productions for v to define the judgment form $e \text{ value}$.

Exercise 0.1. For this part of the lab, implement `eval` for the calculator language above (which is the same language as in the previous lab) but now with type `(Env, Expr) => Expr`.

```
def eval(env: Env, e: Expr): Expr = ???
```

You will not use the `env` parameter yet. You will want to check that your implementation returns JavaScripty values. For example,

Coercions: Basic Values

Booleans, Strings, and Undefined

Like most other languages, JavaScript has other basic value types. Let us extend JavaScripty with Booleans, strings, and the **undefined** value:

$$\begin{array}{ll} \text{values} & v ::= b \mid \text{str} \mid \text{undefined} \\ \text{booleans} & b \\ \text{strings} & \text{str} \end{array}$$

Boolean values b are the literals **true** and **false**. String values str are string literals like **"hi"** that we do not explicitly define here.

The **undefined** literal is a distinguished value that is different than all other values. It is like the unit literal $()$ in Scala.

```
case class B(b: Boolean) extends Expr // e ::= b
case class S(str: String) extends Expr //      | str
case object Undefined extends Expr    //      | undefined
```

Examples

```
true
false
"Hello"
"Hola"
undefined
```

We update our `isValue` function appropriately:

```
def isValue(e: Expr): Boolean = e match {
  case N(_) | B(_) | S(_) | Undefined => true
  case _ => false
}
```

A good exercise here is to reflect on what this code translates to in terms of additional inference rules for the e value judgment form.

Expressions

Each value type comes with some operations. Our abstract syntax tree has two constructors for unary and binary expressions that are parametrized by unary *uop* and binary *bop* operators, respectively:

$$\text{expressions } e ::= | uop\ e_1 | e_1\ bop\ e_2$$

Numbers

For example, numbers has

$$\begin{array}{ll} \text{unary operators } uop & ::= - \\ \text{binary operators } bop & ::= + | - | * | / \end{array}$$

that we considered previously.

Booleans

For booleans, we add unary negation `!` and binary conjunction `&&` and disjunction `||`.

$$\begin{array}{ll} \text{unary operators } uop & ::= ! \\ \text{binary operators } bop & ::= \&\& | || \end{array}$$

```
case object Not extends Uop // uop ::= !
case object And extends Bop // bop ::= &&
case object Or extends Bop  //      ||
```

Examples

```
!true
true && false
true || false
```

We also expect to be able to eliminate booleans with a conditional if-then-else expression:

$$\text{expressions } e ::= e_1 ? e_2 : e_3$$

```
case class If(e1: Expr, e2: Expr, e3: Expr) extends Expr // e ::= e1 ? e2 : e3
```

We also expect to be able to compare values for equality and disquality and numbers for inequality:

binary operators $bop ::= == \mid != \mid < \mid <= \mid > \mid >=$

```
case object Eq extends Bop // bop ::= ==
case object Ne extends Bop //      | !=
case object Lt extends Bop //      | <
case object Le extends Bop //      | <=
case object Gt extends Bop //      | >
case object Ge extends Bop //      | >=
```

Strings

The string operations we support in JavaScripty are string concatenation and string comparison. In JavaScript, string concatenation is written with the binary operator `+` and string comparison using `<`, `<=`, `>`, and `>=`, so we do not need to extend the syntax.

Examples

```
"Hello" + ", " + "World" + "!"
```

Undefined

As **undefined** corresponds to unit `()` in Scala and is uninteresting in itself, we add a side-effecting expression that prints to console.

expressions $e ::= \text{console.log}(e_1)$

```
case class Print(e1: Expr) extends Expr // e ::= console.log(e1)
```

Examples

```
console.log("Hello, World!")
```

If we now have side-effecting expressions, then we would expect to have a way to sequence executing expressions for their effects.

binary operators $bop ::= ,$

```
case object Seq extends Bop // bop ::= ,
```

Examples

```
undefined, 3
```

Semantics Detective: JavaScript is Bananas

In the above, we have carefully specified the abstract syntax of the object language of interest and informally discussed its semantics. But if we are to implement an interpreter in the `eval` function, we also need to define its semantics! And we give a precise definition as follows:

Important

In this lab, the semantics of a JavaScripty expression e is defined by the evaluation of it as a JavaScript program.

Given the careful specification of the abstract syntax, a natural question to ask is whether all abstract syntax trees of type `Expr` in the above are valid expressions and have a semantics in JavaScript (and hence JavaScripty). Is `3 + true` a valid expression?

```
//| filename: JavaScript
3 + true
```

We try it out and see that yes it is. One aspect that makes the JavaScript specification interesting is the presence of implicit coercions (e.g., non-numeric values, such as booleans or strings, may be implicitly converted to numeric values depending on the context in which they are used).

You might guess that defining coercions between value types can lead to some interesting semantics. It is because of these coercions that we have the meme that “JavaScript is bananas.”

```
//| filename: JavaScript
"b" + "a" + "n" + - "a" + "a" + "s"
```

Armed with knowledge that in JavaScript, numbers are floating-point numbers, the `+` operator in JavaScript is overloaded for strings and numbers, and coercions happen between value types, see if you can explain what is happening in the “bananas” expression above.

Coercions

Our `eval` function interpreter will need to make use of three helper functions for converting values to numbers, booleans, and strings:

```
def toNumber(v: Expr): Double = ???  
def toBoolean(v: Expr): Boolean = ???  
def toStr(v: Expr): String = ???
```

Exercise 0.2. Write at least 1 JavaScript expression that shows a coercion from a non-numeric value to a number and see what the result should be:

```
///  
// filename: JavaScript  
// YOUR CODE HERE  
undefined
```

Then, translate this JavaScript expression (written in concrete syntax) into an `Expr` value (i.e., a JavaScripty abstract syntax tree).

Finally, use this `Expr` as a unit test for `toNumber` in the `Lab2StudentSpec` class in the `Lab2Spec.scala` file.

Listing 1 Lab2Spec.scala

```
val e_toNumber_test1 = ???  
"toNumber" should s"${e_toNumber_test1}" in {  
  assertResult(???) { toNumber(e_toNumber_test1) }  
}
```

Exercise 0.3. Do the same to create a `toBoolean` test. Write at least 1 JavaScript expression that shows a coercion from a non-boolean value to a boolean, see what the result should be, translate it to an `Expr` value, and add it as test in `Lab2StudentSpec`.

```
///  
// filename: JavaScript  
// YOUR CODE HERE  
undefined
```

Exercise 0.4. Do the same to create a `toStr` test. Write at least 1 JavaScript expression that shows a coercion from a non-string value to a string, see what the result should be, translate it to an `Expr` value, and add it as test in `Lab2StudentSpec`.


```

//| filename: JavaScript
// YOUR CODE HERE
undefined

```

Exercise 0.5. Implement `toNumber`, `toBoolean`, and `toStr` in `Lab2.scala` (using test-driven development with the test cases you have written above).

Exercise 0.6. For this part of the lab, extend your `eval` from Exercise 0.1 for booleans, strings, the undefined value, and printing.

```
def eval(env: Env, e: Expr): Expr = ???
```

You still will not use the `env` parameter yet. You again will want to check that your implementation returns JavaScripty values using the latest `isValue`. For example,

Interpreter: JavaScripty Variables

The final piece of this lab is to extend our interpreter with variable uses and binding (cf. `?@sec-static-scoping`).

expressions $e ::= x \mid \text{const } x = e_1; e_2$

```

case class Var(x: String) extends Expr // e ::= x
case class ConstDecl(x: String, e1: Expr, e2: Expr) extends Expr // | const x = e1; e2

```

Note that the above is the abstract syntax we consider for `ConstDecl`, which is more flexible than the concrete syntax for `const` allowed by JavaScript (cf. `?@sec-javascripty-concrete-syntax-declarations`).

In this lab, we define a value environment as a map from variable names to JavaScripty values, which we represent in Scala as a value of type `Map[String, Expr]`. Note that representing variable names as Scala `Strings` here, and it is a representation invariant that the `Exprs` must correspond to JavaScripty values.

```

type Env = Map[String, Expr]
val empty: Env = Map.empty

def lookup(env: Env, x: String): Expr = env(x)
def extend(env: Env, x: String, v: Expr): Env = {
  require(isValue(v))
  env + (x -> v)
}

```

We provide the above value and function bindings to interface with the Scala standard library for `Map[String, Expr]` and maintain this representation invariant. You may use the Scala standard library directly if you wish, but we recommend that you just use these interfaces, as they are all that you need and give you the safety of enforcing the representation invariant. The `empty` value represents an empty value environment, the `lookup` function gets the value bound to the variable named by a given string, and the `extend` function extends a given environment with a new variable binding.

Exercise 0.7. For this part of the lab, extend your `eval` from Exercise 0.1 for variable uses (i.e., `Var`) and variable binding (i.e., `ConstDecl`). We suggest you start by adding tests with variables uses and `const` bindings to be able to do test-driven development (see below).

```
def eval(env: Env, e: Expr): Expr = ???
```

Testing

In this lab, we have carefully defined the syntax of the JavaScripty variant of interest, and we have defined its semantics to be defined to be the same as JavaScript. Thus, you can write any JavaScript program within the syntax defined above to create test cases for your `eval` function. Any of the above JavaScript examples could be used as test cases. In some cases, you may want to write abstract syntax trees directly in Scala (i.e., values of type `Expr`). In other cases, you can use the provided JavaScripty parser to translate concrete syntax (i.e., values of type `String`) into abstract syntax (i.e., values of type `Expr`).

Exercise 0.8 (Optional). We give some exercises below to explore this subset of JavaScripty that you can then use as test cases that you add to your `Lab2StudentSpec`.

Basic Arithmetic Operations

This program defines two constants, `x` and `y`, and calculates their sum. It then logs the result `sum` to the console.

```
const x = 10;
const y = 5;
const sum =
  // YOUR CODE HERE (replace undefined)
  undefined
;
console.log(sum);
```

```
console.assert(sum === 15)
```

Conditional Expressions

```
const a = 10;  
const b = 20;  
const max =  
  // YOUR CODE HERE (replace undefined)  
  undefined  
;
```

```
console.assert(max === 20)
```

Unary and Binary Operations

This program checks if a number is positive using a unary negation `-` and a binary relational operator.

```
const num = -5;  
const isPositive =  
  // YOUR CODE HERE (replace undefined)  
  undefined  
;
```

```
console.assert(isPositive === false)
```

Undefined

This program demonstrates the correspondence between `undefined` in JavaScript and `()`.

```
const r = console.log("Hello");
```

```
console.assert(r === undefined)
```

Submission

If you are a University of Colorado Boulder student, we use Gradescope for assignment submission. In summary,

- ☐ Create a private GitHub repository by clicking on a GitHub Classroom link from the corresponding Canvas assignment entry.
- ☐ Clone your private GitHub repository to your development environment (using the <> Code button on GitHub to get the repository URL).
- ☐ Work on this lab from your cloned repository. Use Git to save versions on GitHub (e.g., `git add`, `git commit`, `git push` on the command line or via VSCode).
- ☐ Submit to the corresponding Gradescope assignment entry for grading by choosing GitHub as the submission method.

You need to have a GitHub identity and must have your full name in your GitHub profile in case we need to associate you with your submissions.